



INDIA.RUNS

Build what next India runs on

Team Name : Argmax

Team Leader Name : Sathya T

Problem Statement : Rank the top-100 best-fit candidates from a 100,000-profile pool for the Senior AI Engineer (Founding Team) @ Redrob AI role - contextual + behavioral fit over keyword overlap, CPU-only and offline (≤ 5 min).

Solution Overview

What is your proposed solution?

- A hybrid, explainable, CPU-only ranking engine: honeypot filter -> JD gated rubric -> semantic + lexical relevance -> cross-encoder rerank -> behavioral multiplier -> fact-grounded reasoning.

What makes it different from keyword matching?

- Substance over keywords: scores what candidates BUILT (career history); the skills list is excluded, so keyword-stuffers don't rank.
- Reads profiles: an impossibility/honeypot filter removes logically-impossible profiles (0 in our top-100 vs the >10% that disqualifies).
- Availability-aware: a bounded behavioral multiplier down-weights unreachable 'perfect-on-paper' candidates.
- Compliant by design: heavy models run offline; the scored step is stdlib, CPU-only, offline, ~190s.

JD Understanding & Candidate Evaluation

Key requirements extracted from the JD

- Must-have: production embeddings retrieval, vector/hybrid search, strong Python, ranking evaluation (NDCG/MRR/MAP/A-B).
- Ideal profile: 5-9 yrs, product company (not pure services/research), shipped an end-to-end ranking/search/recsys system, Pune/Noida.
- Negatives: services-only, CV/speech-without-NLP, title-chasers, recent-LLM-only.

Evaluating fit beyond keyword matching

- Career-evidence of retrieval/ranking work + an engineering-role gate ('Marketing Manager' + AI skills ~ 0).
- Skill DEPTH (proficiency x endorsements/duration x assessments), not count; embeddings catch plain-language Tier-5s; 23 Redrob signals weigh availability.

Ranking Methodology

Retrieve -> score -> rerank

- Recall funnel: a fast structured score over all 100k keeps a top pool.
- Semantic: bge-small-en-v1.5 cosine to a distilled JD query, blended 50/50 with lexical concept matching.
- Rerank: bge-reranker-v2-m3 cross-encoder takes half-authority over the top tier (drives NDCG@10).
- Weighted components: relevance .30, title/career .28, skill-depth .20, experience .12, education .10.
- Then multiply by soft penalties (services/CV-speech/title-chaser/location) and a bounded behavioral availability factor (0.5 to 1.15).
- Final: score = fit x penalties x behavioral; tie-break by candidate_id. Models run offline; rank.py reads cached JSON.

Explainability & Data Validation

How decisions are explained

- Each candidate shows a component breakdown + relevance parts (lexical / semantic / cross-encoder) and a 1-2 sentence reasoning.

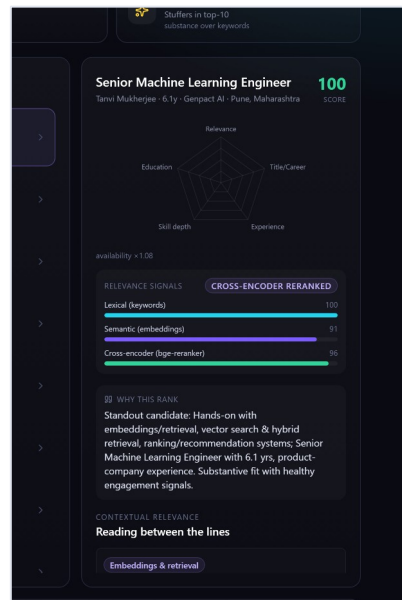
No hallucinations

- Reasoning is templated strictly from real profile facts (no LLM), so it cannot cite anything not in the profile; tone matches rank.

Suspicious profiles

- Honeypot filter (e.g. 'expert' skill with 0 months used). 65 flagged, 0 false positives, 0 reach the top-100.

Explainability in the demo UI



End-to-End Workflow

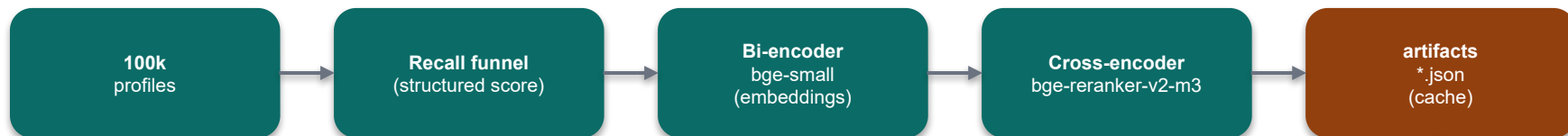
Complete workflow: JD input -> ranked output

- OFFLINE (GPU ok, one-time): embed the candidate pool + cross-encoder rerank -> cache artifacts/*.json.
- SCORED (rank.py, CPU/offline, <=5 min): stream 100k -> honeypot filter -> component scoring x behavioral multiplier -> top-100 heap -> fact-grounded reasoning -> submission.csv.
- Validate: data/validate_submission.py confirms format before upload.

Full workflow diagram: docs/05_approach_and_roadmap.md

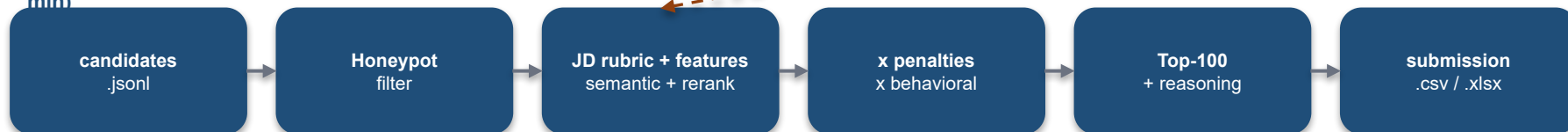
System Architecture

1 · OFFLINE PRECOMPUTE (GPU, one-time)



cached scores -> loaded by rank.py

2 · SCORED RANKING - rank.py (CPU · offline · <=5 min)



Demo sandbox: app/ (FastAPI) + frontend/ (React + Tailwind) -> single Docker image (HF Space), reading the same src/. Compliance: GPU / transformers live only in precompute; rank.py is stdlib · CPU · offline.

Results & Performance

0

honeypots in top-100

~190s

CPU · offline run

83/100

in the 5-9 yr band

97/100

show ranking work

```
$ just check          # rank.py -> submission.csv, then validate
```

Scored 100,000 candidates (65 honeypots filtered) in 186.8s; top score 1.03, cutoff 0.77

Wrote 100 ranked candidates to submission.csv Submission is valid.

```
$ uv run python scripts/quality_proxy.py submission.csv
```

top-100: in-band 83/100 ranking-work 97/100 juniors 1/100 available 92/100

Top-10: senior product-company retrieval/ranking/NLP engineers (Meta, Apple, Netflix, Zomato, Paytm, Razorpay). Ground truth hidden - validated by ablation + calibration (docs/09).

Technologies Used

Technologies & why

- Ranker: Python 3.13, stdlib-only - deliberate, guarantees CPU/offline reproducibility at Stage 3.
- Offline precompute: PyTorch + sentence-transformers; bge-small-en-v1.5 (embeddings) + bge-reranker-v2-m3 (cross-encoder) - open-source, no hosted API (Cohere excluded: network).
- Demo: FastAPI; Vite + React + TypeScript + Tailwind; Recharts.
- Tooling: uv (reproducible envs, no pip); just (cross-platform tasks); Docker (HF Space).

Every choice keeps the scored path inside the compute budget while heavy models run offline.

Submission Assets

Submission assets

- GitHub (public): full code + docs; reproduce with just rank.
- Ranked output: submission.xlsx (top-100).
- This deck (PDF): approach, methodology, results.
- README + docs/00-09 + PROJECT_LOG (iteration trail).
- Live sandbox: hosted Hugging Face Docker Space (also: just serve / docker-run).

LIVE on Hugging Face Space

The screenshot shows a Hugging Face Docker Space interface with a dark theme. At the top, there are two status indicators: 'Homonyms caught: found out of top-100' and 'Top-10 differ vs keyword: only 5 overlap'. Below this, the section 'RANKED OUTPUT' is visible, followed by 'Top candidates (70)'. A list of candidates is displayed, each with a rank, a role, a score, and a 'top 10' badge. The roles include Senior Machine Learning Engineer, Staff Machine Learning Engineer, Lead AI Engineer, Search Engineer, Senior NLP Engineer, Senior AI Engineer, and AI Engineer. Each entry also lists the candidate's name and location, and the methods used for ranking: 'Embeddings & retrieval', 'Vector DB / hybrid search', and 'Ranking & recommendation'.

Rank	Role	Score	Top 10	Methods
1	Senior Machine Learning Engineer	8.1y	Yes	Embeddings & retrieval, Vector DB / hybrid search, Ranking & recommendation
2	Senior Machine Learning Engineer	7.2y	Yes	Embeddings & retrieval, Vector DB / hybrid search, Ranking & recommendation
3	Staff Machine Learning Engineer	7.0y	Yes	Embeddings & retrieval, Vector DB / hybrid search, Ranking & recommendation
4	Lead AI Engineer	6.1y	Yes	Embeddings & retrieval, Vector DB / hybrid search, Ranking & recommendation
5	Search Engineer	7.8y	Yes	Embeddings & retrieval, Vector DB / hybrid search, Ranking & recommendation
6	Senior NLP Engineer	7.8y	Yes	Embeddings & retrieval, Vector DB / hybrid search, Ranking & recommendation
7	Senior AI Engineer	5.9y	Yes	Embeddings & retrieval, Vector DB / hybrid search, Ranking & recommendation
8	AI Engineer	4.8y	Yes	Embeddings & retrieval, Vector DB / hybrid search, Ranking & recommendation

live: tsathya98-redrob-candidate-ranker.hf.space



INDIA.RUNS

Build what next India runs on

THANK YOU